

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 06 -

PROGRAMAÇÃO ORIENTADA A OBJETOS (POO) EM PROJETOS DE ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS.....	20

EMSE

O QUE VEM POR AÍ?

Projetos de Machine Learning vão muito além de treinar modelos: exigem código bem estruturado, extensível e sustentável. Esta aula conduz você da motivação ao uso prático da Programação Orientada a Objetos em ML, mostrando como evoluir de scripts simples para sistemas bem projetados.

Serão revisitados os fundamentos de POO com foco em aplicações reais, conectando conceitos como encapsulamento, abstração e polimorfismo ao design de pipelines de ML. Exploraremos decisões essenciais de arquitetura, como herança versus composição, e padrões de projeto amplamente utilizados na indústria.

Também serão discutidas boas práticas de código, integração entre paradigmas OO e funcional e estratégias para evitar complexidade desnecessária. Ao final, você estará preparado(a) para enxergar projetos de Machine Learning como sistemas de software completos e prontos para escalar.

HANDS ON

Projetos de Machine Learning evoluem rapidamente de scripts exploratórios para sistemas mais complexos, o que torna a organização do código um fator essencial para manutenção e escalabilidade. Nesse contexto, a Programação Orientada a Objetos é apresentada como uma abordagem capaz de estruturar projetos de ML de forma mais clara, reutilizável e sustentável ao longo do tempo.

Nessa aula, discutiremos as motivações para o uso de POO em Machine Learning, destacando limitações de código não estruturado e a importância de boas decisões de design. Em seguida, serão revisitados os conceitos fundamentais da Programação Orientada a Objetos, como classes, objetos, atributos e métodos, além de princípios como encapsulamento, abstração, herança e polimorfismo, sempre conectando esses conceitos ao domínio de ML.

Na sequência, será apresentado o design orientado a objetos aplicado a projetos de Machine Learning, enfatizando a separação de responsabilidades e a organização do sistema em componentes como pipelines de dados, modelos e avaliadores. Também serão discutidas estratégias para modelar modelos de ML como objetos, adotando interfaces padronizadas que facilitam reutilização, testes e evolução do sistema.

O material abordará ainda a escolha entre herança e composição, destacando a composição como uma alternativa mais flexível em muitos cenários de Machine Learning. Padrões de projeto orientados a objetos, como Strategy e Factory, são introduzidos para ilustrar como o polimorfismo pode ser utilizado para estender comportamentos de forma controlada. Por fim, serão discutidas boas práticas de código, a integração entre Programação Orientada a Objetos e Programação Funcional e uma atividade prática de desenho de classes, reforçando a importância do planejamento e do design na construção de sistemas de ML robustos e escaláveis.

SAIBA MAIS

Projetos de Machine Learning raramente começam como sistemas completos e bem estruturados. Na maioria dos casos, eles se iniciam como scripts exploratórios, criados para analisar dados, testar hipóteses ou validar rapidamente um modelo. Esse tipo de abordagem é natural e até desejável nas fases iniciais, pois favorece agilidade e experimentação. No entanto, à medida que o projeto evolui, esses scripts tendem a crescer em tamanho e complexidade, incorporando novas etapas de pré-processamento, diferentes versões de modelos, ajustes de hiperparâmetros, métricas adicionais e integrações com outros sistemas.

Esse crescimento gradual expõe as limitações do código não estruturado. Scripts extensos, compostos por funções soltas e variáveis compartilhadas, tornam-se difíceis de compreender, modificar e manter. Pequenas alterações podem gerar efeitos inesperados em partes distantes do código e a reutilização de componentes passa a exigir cópias e adaptações manuais. Em projetos maiores, essa falta de estrutura compromete não apenas a produtividade, mas também a confiabilidade dos resultados, uma vez que erros se tornam mais difíceis de se detectar e corrigir.

Nesse contexto, a organização do código deixa de ser uma preocupação estética e passa a ser um fator determinante para a manutenção e a escalabilidade do sistema. Código bem estruturado facilita a leitura, a depuração e a extensão do sistema ao longo do tempo. Em projetos de Machine Learning, em que experimentos são constantemente repetidos e comparados, a capacidade de organizar responsabilidades, isolar componentes e reaproveitar soluções torna-se essencial. A ausência dessa organização frequentemente resulta em sistemas frágeis, difíceis de evoluir e altamente dependentes de conhecimento tácito dos indivíduos desenvolvedores originais.

A Programação Orientada a Objetos (POO) surge como uma resposta a esses desafios ao oferecer um conjunto de conceitos e princípios voltados à modelagem de sistemas complexos. Em vez de organizar o código apenas em funções e variáveis, a POO propõe estruturar o sistema em torno de objetos, que combinam dados (estado) e comportamentos (métodos) relacionados. Essa abordagem permite representar conceitos do domínio de Machine Learning — como modelos, pipelines, avaliadores

e componentes de pré-processamento — de forma mais próxima à realidade do problema.

Ao utilizar POO, torna-se possível encapsular responsabilidades específicas em classes bem definidas, reduzindo o acoplamento entre diferentes partes do sistema. Isso facilita a substituição de componentes, a extensão de funcionalidades e a adaptação do código a novos requisitos. Além disso, a POO favorece a criação de interfaces padronizadas, o que é particularmente relevante em Machine Learning, onde diferentes modelos podem compartilhar operações semelhantes, como treinamento, predição e avaliação.

Dentro do ciclo de vida de projetos de Machine Learning, a Programação Orientada a Objetos ocupa um papel estratégico principalmente nas fases em que o projeto deixa de ser puramente experimental e passa a exigir maior robustez. Enquanto abordagens mais simples podem ser suficientes para análises exploratórias iniciais, a POO torna-se especialmente valiosa na construção de pipelines reutilizáveis, no gerenciamento de múltiplos modelos, na integração com sistemas externos e na preparação do código para uso em produção. Dessa forma, a Programação Orientada a Objetos não substitui outras abordagens, mas complementa o desenvolvimento em Machine Learning, oferecendo uma base sólida para a construção de sistemas mais organizados, escaláveis e sustentáveis ao longo do tempo.

A Programação Orientada a Objetos parte da premissa de que sistemas complexos podem ser mais bem compreendidos quando organizados em torno de entidades que representam elementos do mundo real ou do domínio do problema. Cada uma dessas entidades possui informações próprias e comportamentos associados, formando unidades coerentes de software.

Classes e objetos: do conceito abstrato à instância concreta

O ponto de partida da POO é a distinção entre classe e objeto. Uma classe pode ser entendida como uma descrição abstrata, um modelo conceitual que define como algo deve ser estruturado. Um objeto, por sua vez, é a materialização dessa descrição, isto é, uma instância concreta criada a partir da classe.

Uma analogia visual útil é a de um formulário em branco e um formulário preenchido. O formulário define quais campos existem (classe), enquanto cada formulário preenchido representa um caso específico (objeto).

Em Machine Learning, essa distinção aparece com frequência. Um modelo genérico pode ser descrito por uma classe, enquanto cada modelo treinado com dados específicos corresponde a um objeto distinto.

Uma vez compreendida a ideia de objeto, é importante entender que todo objeto possui duas dimensões complementares: estado e comportamento. O estado é representado pelos atributos, que armazenam informações internas do objeto; o comportamento é representado pelos métodos, que definem as ações que o objeto pode executar.

Uma analogia visual bastante intuitiva é a de um controle remoto. Ele possui informações internas (estado) e botões que executam ações (comportamentos). O usuário não altera diretamente o circuito interno; ele interage por meio das ações disponíveis. Em projetos de ML, essa união entre dados e comportamento ajuda a manter o código mais organizado e menos propenso a inconsistências.

À medida que sistemas crescem, torna-se perigoso permitir que qualquer parte do código altere livremente o estado interno dos objetos. É nesse ponto que entra o encapsulamento, um dos princípios centrais da POO.

Encapsular significa controlar o acesso aos detalhes internos de um objeto. Visualmente, podemos imaginar uma caixa preta: sabe-se o que entra e o que sai, mas não é necessário (nem desejável) conhecer todos os detalhes internos.

Em Machine Learning, isso é especialmente importante para evitar que o estado de um modelo treinado seja modificado de forma indevida.

Enquanto o encapsulamento protege detalhes internos, a abstração atua em um nível mais conceitual. Abstrair significa modelar apenas o que é relevante, deixando de lado detalhes que não são necessários para quem utiliza o objeto.

Uma analogia clássica é a de dirigir um carro. O motorista sabe acelerar, frear e virar o volante, mas não precisa entender o funcionamento interno do motor. Essa simplificação torna o sistema utilizável.

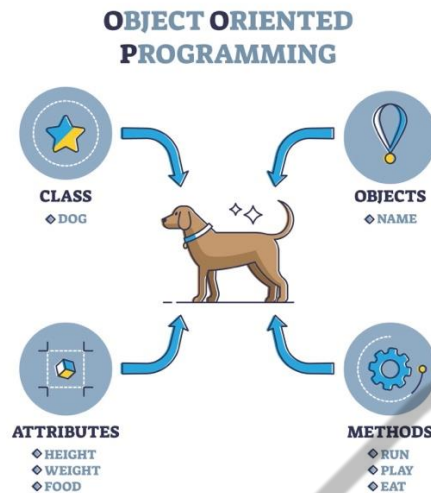


Figura 1 - Programação orientada a objetos ou POO
Fonte: Manujachelaka (2025)

A herança permite criar classes mais específicas a partir de uma classe mais geral. Visualmente, esse conceito é frequentemente representado como uma hierarquia, semelhante a uma árvore genealógica. Em ML, isso permite criar famílias de modelos que compartilham uma interface comum, mas implementam algoritmos diferentes.

O polimorfismo complementa a herança e a abstração ao permitir que objetos diferentes respondam ao mesmo método de maneiras distintas. A ideia central é que o código que utiliza o objeto não precisa saber qual é sua classe concreta.

Uma analogia visual simples é a de um botão “imprimir”: o comando é o mesmo, mas o resultado depende do tipo de documento.

Por fim, os princípios SOLID funcionam como um conjunto de orientações para evitar erros comuns no design orientado a objetos. Dois deles merecem atenção especial neste momento:

- Single Responsibility Principle (SRP): cada classe deve ter um único propósito bem definido.
- Open/Closed Principle (OCP): classes devem poder ser estendidas sem precisar serem modificadas.

Esses princípios ajudam a manter sistemas de ML organizados, evitando classes grandes, confusas e difíceis de manter.

Design Orientado a Objetos aplicado a Projetos de Machine Learning

Após a revisão dos conceitos fundamentais da Programação Orientada a Objetos, o próximo passo natural é compreender como esses conceitos se materializam no design de sistemas de Machine Learning. Até aqui, discutiu-se o que são classes, objetos, encapsulamento e abstração. Agora, o objetivo é responder a uma pergunta central para qualquer projeto real: como estruturar o código de ML de forma organizada, compreensível e sustentável ao longo do tempo?

Em projetos pequenos, é comum que todo o processamento — leitura de dados, limpeza, treinamento e avaliação — esteja concentrado em um único script ou em poucas funções. No entanto, à medida que o projeto cresce, essa abordagem se torna frágil. O design orientado a objetos surge como uma forma de organizar responsabilidades, permitindo que cada parte do sistema tenha um papel claro e bem delimitado.

Um princípio essencial da Programação Orientada a Objetos, já introduzido anteriormente, é o Single Responsibility Principle (SRP): cada classe deve ter uma única responsabilidade. Em Machine Learning, isso significa evitar classes ou módulos que “fazem de tudo”, como carregar dados, treinar modelos e avaliar métricas ao mesmo tempo.

Uma analogia visual útil é a de uma linha de produção. Em vez de um único trabalhador realizar todas as tarefas, cada etapa do processo é atribuída a um posto específico. Essa divisão torna o sistema mais eficiente, mais fácil de ajustar e menos propenso a erros.

Em muitos projetos de Machine Learning, independentemente do domínio, surge uma estrutura conceitual recorrente. Essa estrutura pode ser representada por classes que refletem as etapas principais do pipeline. Uma decomposição típica inclui:

- Uma classe responsável pelo **pipeline de dados**.
- Uma classe responsável pelo **treinamento do modelo**.
- Uma classe responsável pela **avaliação dos resultados**.

Visualmente, pode-se imaginar essas classes como blocos independentes, conectados por interfaces claras, em vez de um único bloco monolítico difícil de manter.

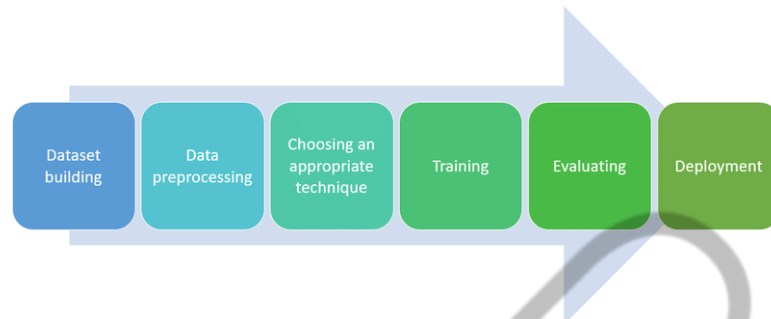


Figura 2 – Classes que refletem as etapas principais do pipeline
Fonte: Hamza (2021)

Ao projetar classes como `DataPipeline`, `ModelTrainer` e `ModelEvaluator`, o objetivo não é apenas dividir o código, mas organizar o conhecimento do sistema. Cada classe passa a encapsular uma parte específica do processo de ML.

- `DataPipeline` concentra tudo o que diz respeito à leitura, limpeza e transformação dos dados.
- `ModelTrainer` lida exclusivamente com o treinamento do modelo e o gerenciamento de seus parâmetros.
- `ModelEvaluator` é responsável por métricas, validação e análise de desempenho.

Essa separação facilita a leitura do código, pois o nome da classe já comunica sua função. Além disso, mudanças em uma etapa do pipeline tendem a impactar apenas a classe correspondente, reduzindo o risco de efeitos colaterais.

Modelando um modelo de ML como objeto

Um passo fundamental no design orientado a objetos aplicado a Machine Learning é tratar o modelo como um objeto, não apenas como uma função matemática. Esse objeto possui estado interno, como hiperparâmetros e parâmetros aprendidos, e comportamentos definidos, como treinar, prever e avaliar. Essa

modelagem aproxima o código da forma como o problema é concebido, permitindo representar o modelo como uma entidade com identidade e ciclo de vida próprios.

Essa abordagem é amplamente utilizada em bibliotecas como o scikit-learn, que padroniza interfaces por meio de métodos como fit e predict. Essa padronização permite utilizar diferentes modelos de forma intercambiável, reforçando princípios como abstração e polimorfismo.

A aplicação da Programação Orientada a Objetos em Machine Learning traz benefícios importantes, como melhor organização do código, maior reutilização de componentes, redução de acoplamento entre etapas, facilidade de teste e manutenção e maior capacidade de evolução do sistema. Esses benefícios são especialmente relevantes em projetos de médio e grande porte, onde múltiplos modelos e experimentos coexistem.

Design OO como base para extensibilidade

Outro aspecto central da POO em Machine Learning é a extensibilidade. Ao organizar o sistema em classes com responsabilidades claras e interfaces bem definidas, torna-se possível adicionar novos modelos, métricas ou etapas de processamento sem modificar o restante do sistema.

Esse princípio está diretamente relacionado ao Open/Closed Principle, segundo o qual um sistema deve estar aberto para extensão, mas fechado para modificação. Isso permite evoluir o sistema sem comprometer experimentos existentes ou introduzir instabilidade.

Mais do que simplesmente utilizar classes, o design OO consiste em organizar o sistema em torno de responsabilidades claras e abstrações bem definidas. Ao modelar pipelines, modelos e avaliadores como objetos que interagem por interfaces, cria-se uma base sólida para sistemas mais robustos, escaláveis e compreensíveis.

Herança versus Composição em Projetos de Machine Learning

Uma decisão fundamental no design orientado a objetos é a escolha entre herança e composição. Ambas permitem reutilização e organização do código, mas

representam formas diferentes de modelar relações entre classes. A escolha correta é essencial para evitar sistemas rígidos e difíceis de manter.

A herança representa uma relação de especialização, na qual uma classe deriva de outra. Em Machine Learning, isso é útil para modelar famílias de modelos com uma interface comum. Por exemplo, uma regressão linear é um tipo específico de modelo de ML.

A herança permite reutilizar comportamentos e facilita o polimorfismo, possibilitando tratar diferentes modelos de forma uniforme. No entanto, apresenta limitações importantes. Hierarquias profundas podem tornar o sistema rígido e mudanças na classe base podem afetar múltiplas subclasses. Além disso, nem todas as relações entre componentes representam uma especialização conceitual válida.

Em projetos de ML, onde pipelines e experimentos mudam com frequência, o uso excessivo de herança pode dificultar a evolução do sistema.

A composição é uma alternativa mais flexível, baseada na ideia de que uma classe possui outras classes e delega a elas parte de seu comportamento. Essa abordagem é particularmente adequada para pipelines de Machine Learning, nos quais diferentes componentes cooperam para produzir um resultado final.

A composição permite substituir ou modificar componentes de forma independente, reduzindo o acoplamento e facilitando a experimentação. Essa flexibilidade é essencial em ML, onde é comum trocar algoritmos, métricas ou estratégias de pré-processamento.

Uma boa analogia é a de blocos de montagem: diferentes combinações produzem sistemas distintos, sem a necessidade de alterar toda a estrutura.

Algumas diretrizes práticas ajudam nessa decisão:

- Utilize herança quando houver uma relação clara e estável do tipo “é-um”.
- Prefira composição quando precisar combinar comportamentos de forma flexível.
- Evite hierarquias profundas.
- Em caso de dúvida, prefira composição.

Ambas podem coexistir. A herança pode definir interfaces comuns, enquanto a composição organiza pipelines completos. Essa combinação oferece padronização e flexibilidade.

Padrões de Projeto Orientados a Objetos em Machine Learning

À medida que sistemas de ML crescem, organizar o código em classes não é suficiente. Surgem desafios como trocar algoritmos, controlar a criação de objetos e estender comportamentos sem modificar o sistema. Os padrões de projeto oferecem soluções estruturadas para esses problemas.

Padrões de projeto são soluções conceituais reutilizáveis para problemas recorrentes de design. Eles ajudam a criar sistemas mais flexíveis, extensíveis e fáceis de manter.

O polimorfismo permite que diferentes objetos respondam à mesma interface de formas distintas. Em Machine Learning, isso permite utilizar diferentes modelos, métricas ou estratégias de forma intercambiável, sem alterar o restante do sistema. Esse princípio é a base de muitos padrões de projeto utilizados em ML.

O padrão Strategy é um dos mais relevantes em Machine Learning. Ele permite definir uma família de algoritmos, encapsulá-los em classes separadas e torná-los intercambiáveis em tempo de execução.

A analogia visual clássica é a de um GPS: o destino é o mesmo, mas a estratégia de rota pode variar (mais rápida, mais curta, sem pedágio).

Em ML, diferentes estratégias podem representar:

- Diferentes algoritmos de treinamento.
- Diferentes métricas de avaliação.
- Diferentes técnicas de pré-processamento.

Outro padrão amplamente utilizado é o Factory. Ele centraliza a lógica de criação de objetos, evitando que o restante do sistema precise conhecer detalhes de implementação. A analogia visual é a de uma linha de montagem: o cliente solicita um produto, mas não precisa saber como ele é fabricado internamente. Em Machine

Learning, factories são úteis quando há múltiplos modelos possíveis, escolhidos com base em configuração ou parâmetros externos.

Muitos padrões de projeto já aparecem implicitamente em bibliotecas consolidadas de Machine Learning. A interface fit/predict, por exemplo, explora polimorfismo; a escolha de algoritmos por configuração remete ao padrão Factory; a troca de métricas ou estratégias de validação reflete o padrão Strategy.

Ao reconhecer esses padrões, o(a) aluno(a) passa a enxergar frameworks de ML não como “caixas mágicas”, mas como sistemas bem projetados, baseados em princípios sólidos de engenharia de software.

Boas Práticas de Programação Orientada a Objetos no Código de Machine Learning

Após compreender como a Programação Orientada a Objetos pode ser aplicada ao design de sistemas de Machine Learning e como padrões de projeto ajudam a estruturar soluções flexíveis, é fundamental discutir boas práticas concretas de uso da POO no código. Neste ponto, o foco deixa de ser apenas o que é possível fazer e passa a ser como fazer bem. Em projetos de ML, decisões ruins de design não costumam falhar imediatamente; elas se manifestam com o tempo, na forma de código difícil de manter, testar e evoluir.

Um erro comum entre iniciantes é acreditar que utilizar classes, por si só, já caracteriza um bom design orientado a objetos. No entanto, classes mal definidas podem ser tão problemáticas quanto scripts desorganizados. Por isso, é essencial adotar critérios claros para avaliar a qualidade do design OO.

Em muitos projetos de Machine Learning, o ponto de partida é um conjunto de funções soltas que compartilham dados por meio de variáveis globais ou estruturas passadas de forma implícita. Esse tipo de organização costuma funcionar em estágios iniciais, mas rapidamente se torna frágil conforme o sistema cresce.

Conceitualmente, esse cenário pode ser imaginado como um emaranhado de fios, no qual todas as partes estão conectadas sem uma organização clara. Qualquer modificação exige cuidado excessivo, pois é difícil prever quais partes do sistema serão afetadas.

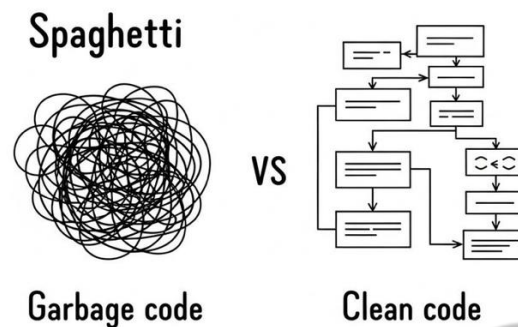


Figura 3 – Código ruim versus código limpo
Fonte: DevTips (2025)

A transição para um design orientado a objetos bem estruturado consiste em identificar responsabilidades e agrupar comportamentos relacionados em classes coesas. O objetivo não é apenas “organizar”, mas tornar o sistema mais compreensível tanto para quem o escreveu quanto para quem irá mantê-lo no futuro.

Uma das boas práticas mais importantes em POO é manter as classes coesas, isto é, focadas em uma única responsabilidade. Esse princípio, já discutido anteriormente como Single Responsibility Principle (SRP), é especialmente relevante em projetos de ML, em que existe a tentação de concentrar todo o fluxo em uma única classe “central”.

Classes que fazem “um pouco de tudo” tendem a crescer rapidamente, acumulando métodos e atributos sem relação direta entre si. Isso dificulta testes, reutilização e entendimento do código. Em contraste, classes menores e especializadas tornam o sistema mais modular e previsível.

Uma boa pergunta orientadora é: se esta classe mudar, quantos motivos legítimos existem para essa mudança? Se a resposta for “muitos”, provavelmente a classe está assumindo responsabilidades demais.

Outra boa prática fundamental é evitar herança quando a relação conceitual não é claramente do tipo “é-um”. Conforme discutido no tópico anterior, herança excessiva tende a gerar estruturas rígidas, difíceis de adaptar às mudanças frequentes típicas de projetos de ML.

A composição, por outro lado, favorece flexibilidade. Ao combinar objetos menores para formar comportamentos mais complexos, o sistema se torna mais fácil

de estender e refatorar. Essa prática reduz o acoplamento e facilita a experimentação, algo central em Machine Learning.

Do ponto de vista pedagógico, é importante reforçar que herança não deve ser vista como o principal mecanismo de reutilização, mas como uma ferramenta específica para casos bem definidos.

Um erro recorrente em projetos orientados a objetos é o excesso de abstrações. Estudantes, ao aprenderem POO, podem sentir-se tentados(as) a criar muitas camadas, interfaces e classes genéricas, mesmo quando o problema não exige esse nível de complexidade.

O princípio KISS (Keep It Simple, Stupid) aplica-se diretamente à POO: abstrações devem existir para resolver problemas reais, não para “embelezer” o código. Em Machine Learning, onde a experimentação rápida é importante, abstrações excessivas podem dificultar ajustes simples e tornar o código mais difícil de entender.

A transformação de um código mal estruturado em um design OO bem definido geralmente resulta em ganhos claros: maior legibilidade; melhor separação de responsabilidades; facilidade de teste; menor risco de efeitos colaterais inesperados; maior capacidade de evolução do sistema.

É importante destacar que refatorar código para POO não significa reescrever tudo do zero, mas organizar progressivamente o que já existe. Essa visão incremental é essencial em projetos de ML, que raramente são construídos de forma linear e definitiva.

Integração entre Programação Orientada a Objetos e Programação Funcional

Em projetos modernos de Machine Learning, a Programação Orientada a Objetos e a Programação Funcional atuam de forma complementar. A Programação Orientada a Objetos é ideal para estruturar o sistema e representar entidades com estado, como modelos, pipelines e avaliadores.

A Programação Funcional é mais adequada para transformações de dados, como limpeza, normalização e cálculo de métricas. Funções puras e imutabilidade tornam essas transformações mais previsíveis e testáveis.

Na prática, é comum que classes utilizem funções puras internamente. Assim, a POO organiza o sistema, enquanto a programação funcional implementa as transformações. Essa separação reduz a complexidade e melhora a clareza do código.

Uma analogia útil é a de uma orquestra: a POO atua como o maestro, organizando o sistema, enquanto a programação funcional executa as transformações específicas.



O QUE VOCÊ VIU NESTA AULA?

Projetos de Machine Learning raramente permanecem simples. O que começa como scripts exploratórios evolui para sistemas mais complexos, que precisam lidar com múltiplas fontes de dados, versões de modelos, métricas, experimentos e integrações externas. A Programação Orientada a Objetos (POO) surge como um paradigma essencial para estruturar projetos de ML de forma mais robusta e sustentável.

Inicialmente, foi discutida a motivação para o uso da POO em Machine Learning, destacando as limitações de códigos não estruturados, como alto acoplamento, dificuldade de manutenção e maior risco de erros.

Em seguida, foram revisitados os conceitos fundamentais da POO. Classes e objetos foram apresentados como formas de modelar entidades do domínio, enquanto atributos representam o estado e métodos representam o comportamento. Princípios como encapsulamento e abstração permitem proteger o estado interno e simplificar o uso do sistema. Já herança e polimorfismo possibilitam reutilização de código e criação de interfaces padronizadas, permitindo que diferentes modelos sejam utilizados de forma intercambiável.

O conteúdo avançou para o design orientado a objetos aplicado especificamente a Machine Learning. Foi enfatizada a importância de organizar o sistema em torno de responsabilidades claras, alinhadas ao princípio da responsabilidade única.

Um ponto central foi a modelagem do modelo de Machine Learning como um objeto, com estado interno e comportamentos definidos, como treinar e prever. Essa abordagem aproxima o código do domínio do problema e favorece a padronização de interfaces, facilitando a substituição de algoritmos e a evolução do sistema.

Também foi discutida a escolha entre herança e composição. A herança é adequada quando existe uma relação clara de especialização (“é-um”), mas pode gerar estruturas rígidas. A composição, baseada em relações do tipo (“tem-um”), mostrou-se mais flexível e adequada para pipelines de ML, permitindo combinar componentes de forma modular e reduzir acoplamento.

Na sequência, foram apresentados padrões de projeto importantes, como Strategy e Factory. Esses padrões permitem variar comportamentos e criar objetos de forma controlada, sem modificar o restante do sistema, reforçando princípios como extensibilidade e baixo acoplamento. Esses conceitos são amplamente utilizados em frameworks reais de Machine Learning. Por fim, foi apresentada a integração entre Programação Orientada a Objetos e Programação Funcional.

Em síntese, esta aula apresentou os fundamentos e aplicações da Programação Orientada a Objetos em Machine Learning, incluindo conceitos básicos, decisões de design, padrões de projeto e boas práticas. Esses conhecimentos fornecem uma base sólida para o desenvolvimento de sistemas de ML mais organizados, escaláveis e manuteníveis, preparando o(a) aluno(a) para desafios reais em ambientes acadêmicos e profissionais.

REFERÊNCIAS

DEVTIPS. **Stop writing code that future devs will hate you for.** 2025. Disponível em: https://dev.to/dev_tips/stop-writing-code-that-future-devs-will-hate-you-for-137j. Acesso em: 25 mar. 2026.

FOWLER, M. **Refactoring: improving the design of existing code.** 2. ed. Boston: Addison-Wesley, 2018.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design patterns: elements of reusable object-oriented software.** Boston: Addison-Wesley, 1994.

HAMZA, B. **The comprehensive programming language model.** 2021. Disponível em: https://www.researchgate.net/publication/353620685_The_comprehensive_programming_language_model. Acesso em: 25 mar. 2026.

MANUJACHELAKA. **Object-Oriented Programming in Game Development.** 2025. Disponível em: <https://medium.com/@manujachelaka/object-oriented-programming-in-game-development-173b52bc3609>. Acesso em: 25 mar. 2026.

MARTIN, R. C. **Agile software development: principles, patterns, and practices.** Upper Saddle River: Prentice Hall, 2003.

MARTIN, R. C. **Clean Architecture: a craftsman's guide to software structure and design.** Boston: Prentice Hall, 2017.

MARTIN, R. C. **Clean Code: a handbook of agile software craftsmanship.** Boston: Prentice Hall, 2008.

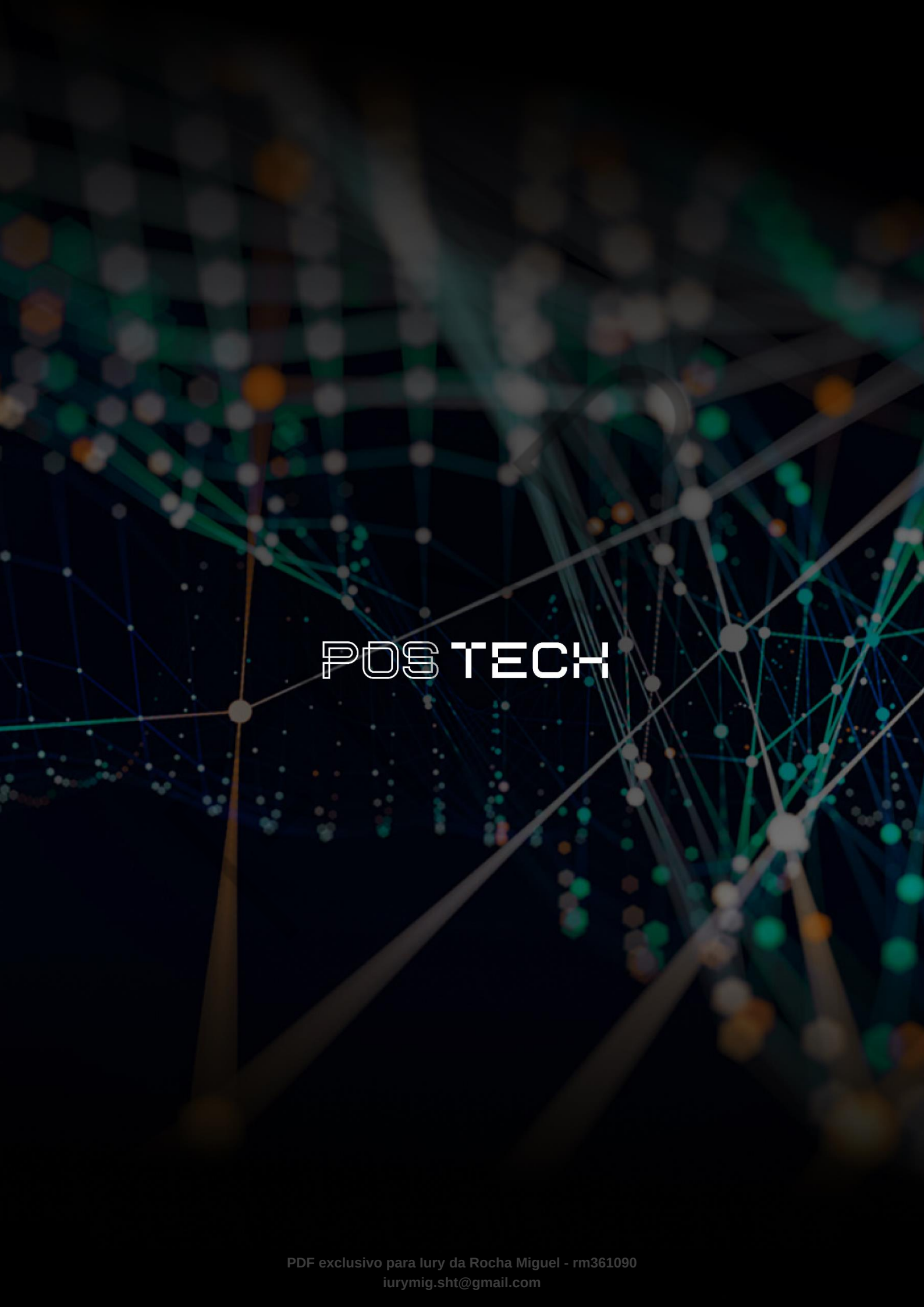
MCLAUGHLIN, B.; POLLICE, G.; WEST, D. **Head First object-oriented analysis and design.** Sebastopol: O'Reilly Media, 2006.

MEYER, B. **Object-oriented software construction.** 2. ed. Upper Saddle River: Prentice Hall, 1997.

PALAVRAS-CHAVE

Arquitetura. Extensibilidade. Responsabilidade.

EMSE



POSTECH